



MAKİNE DİLİ ve BROOKSHEAR MİMARİSİ

Zinha TANTELHARISOA
tarafından sunulmuştur.



Bilgisayar Mimarisi – Giriş



- Bilgisayarın temel bileşenleri
- CPU'nun genel rolü
- Donanım–yazılım ilişkisi



CPU (Central Processing Unit)

- Komutları işler
- Aritmetik ve mantık operasyonlarını yürütür
- Kontrol sinyallerini üretir



CPU Bileşenleri – Genel Bakış

- ALU
- Control Unit
- Register seti
- Veri ve adres yolları

ALU – Aritmetik Mantık Birimi



- Toplama/çıkarma
- Mantıksal işlemler (AND, OR...)
- Karşılaştırma

Control Unit – Kontrol Birimi



- Komutları yorumlar
- Kontrol sinyallerini yönetir
- Makine döngüsünü yönlendirir



Register'lar – Kaydediciler

- CPU içi küçük ve hızlı hafıza
- Geçici veri ve adres tutma
- Program counter, accumulator vb.



Register Türleri

- PC (Program Counter)
- IR (Instruction Register)
- ACC (Accumulator)
- Genel amaçlı register'lar



Makine Dili – Giriş

- En düşük seviye dil
- 0 ve 1'lerden oluşur
- Donanım tarafından doğrudan anlaşılır



Bilgisayarlar Komutları Nasıl Anlar?

- Her komutun ikili bir kodu vardır
- CPU bu kodları çözer
- Her işlem için belirli bit düzeni



Instruction Set – Komut Kümesi

- CPU'nun anlayabildiği tüm komutlar
- Mimariye özel
- Basit veya kompleks olabilir



Komut Yapısı – Op-code

- İşlem türünü belirler
- CPU'ya hangi operasyonu yapacağını söyler

Komut Yapısı – Operand



- İşlenecek veri veya adres
- Sabit değer, register ya da bellek adresi

Op-code + Operand Örneği

- ADD R1 $\rightarrow$ R1'e toplama işlemi
- LOAD 45 $\rightarrow$ 45. adresten veri al



Brookshear Makinesi – Giriş

- Eğitim amaçlı sanal makine
- Basit komut kümesi
- 8-bit odaklı yapı



Brookshear Makinesi – Bellek Yapısı

- 256 hücre
- Her hücre 1 byte
- Program + veri aynı alanda

Brookshear Komutları – 1



- LOAD
- STORE
- MOVE



Brookshear Komutları – 2

- ADD
- SUB
- INPUT
- OUTPUT



Brookshear Komutları – 3

- JUMP
- JUMZ
- JUMNZ



Brookshear Komutları – 4

- HALT
- Diğer temel kontrol komutları



LOAD / STORE Mantığı

- LOAD = Bellekten register'a
- STORE = Register'dan belleğe



Programın Yürütülmesi – Makine Döngüsü

- Fetch – Komutu çek
- Decode – Komutu çöz
- Execute – Komutu çalıştır



Fetch Adımı

- PC komut adresini gösterir
- Bellekten komut okunur
- IR içine yüklenir



Decode Adımı

- Op-code ayrıştırılır
- Operand belirlenir
- Kontrol birimi gerekli sinyalleri üretir



Execute Adımı

- ALU işlemi yapar
- Bellek erişimi olur
- Register'lar güncellenir



Örnek Çalışma Döngüsü

- LOAD $\rightarrow$ ADD $\rightarrow$ STORE örneği
- PC sonraki komutu gösterir

Genel Özet



- CPU = ALU + Control Unit + Register'lar
- Machine language = komut kodları
- Fetch–Decode–Execute = temel döngü